

中小企業向けハイブリッド MLOps : ハイブリッドワーカー実行可能なセルフホスト型軽量 CI/CD 基盤

ヴィルヘルム アルノ* (サボア・モンブラン大学)

増田 嶺 小林 拓実 亀井 仁志 (香川大学)

Hybrid MLOps for SMEs: A Lightweight Self-Hosted CI/CD Infrastructure with Hybrid Worker Execution

Arno Wilhelm*, (Savoie Mont-Blanc University)

Rei Masuda, Takumi Kbayashi, Hitoshi Kamei, (Kagawa University)

Conventional MLOps is extremely expensive for Small and Medium Enterprises (SMEs). We developed Hybrid MLOps, a lightweight cost-efficient CI/CD framework that temporarily delegates Machine Learning (ML) tasks to external clouds instead of on-premises environments. We implemented and evaluated this pipeline in a private cloud, showing significant cost and energy reduction based on task features.

キーワード : 継続的インテグレーション, 機械学習, オンプレミスインフラ, MLOps, モデルベンチマーク評価 (CI/CD, Machine Learning, On-premises Infrastructure, MLOps, Model Benchmarking)

1. Introduction

The rapid expansion of machine learning (ML) has positioned machine learning operations (MLOps) as a central paradigm for ensuring scalability, reproducibility, and automation in ML workflows⁽¹⁾⁽²⁾. However, conventional pipelines often neglect two increasingly critical challenges. First, in practice, ML training is increasingly distributed across heterogeneous environments, including on-premises servers, edge devices, and cloud-based GPUs⁽²⁾⁽³⁾. Second, with the financial and environmental impact of large-scale training steadily increasing, developers require frameworks that treat energy consumption and monetary costs as primary metrics, alongside accuracy, efficiency, and scalability⁽⁴⁾⁽⁵⁾. The absence of such metrics hinders informed decision-making in resource-constrained or sustainability-oriented contexts.

To address these challenges, Hybrid MLOps, first proposed by Hiram et al. in 2023⁽²⁾. This approach extends conventional pipelines through principles such as infrastructure-aware scheduling and fine-grained monitoring of costs and energy. In this paper, we focus on the orchestration layer, which enables developers to declaratively configure and reproduce experiments across heterogeneous infrastructures, while automatically collecting performance, cost, and energy metrics.

We assess the feasibility of this approach through preliminary

experiments on image classification tasks (CNNs and DNNs), comparing GPU-enabled and CPU-only environments and executed it on a small-scale VM infrastructure⁽⁶⁾⁽⁷⁾. These experiments highlight the trade-offs between training speed, financial cost, and energy footprint. Moreover, they show how Hybrid MLOps facilitates transparent and reproducible benchmarking across infrastructures.

This work makes three primary contributions:

- **A YAML-based orchestration structure** for reproducible hybrid MLOps pipelines.
- **An integrated protocol** for systematically measuring performance, financial costs, and energy consumption.
- **An experimental evaluation** illustrating the trade-offs between CPU and GPU environments.

By embedding sustainability and cost-awareness into the orchestration of ML workflows, Hybrid MLOps expands the scope of MLOps beyond operational efficiency, promoting a more transparent, reproducible, and responsible paradigm⁽²⁾⁽⁸⁾.

2. Background

Machine learning (ML) workflows often rely on cloud-based infrastructures for scalability and access to specialized hardware. However, small laboratories or enterprises face challenges such as data sovereignty, security, high operational costs for GPU-intensive tasks, and it is difficult to ensure reproducibility in

dynamic cloud environments.

Hybrid MLOps addresses these challenges by combining local and external resources. A key characteristic of MLOps is that the training phase is temporary and resource-intensive. By executing this task on external cloud services or shared computational resources, sufficient processing power can be secured while confining costs to the training period. Conversely, persistent services such as model deployment, monitoring, and storage remain on-premise. Hosting these services externally incurs continuous costs. These costs can gradually increase over time as data accumulates. In this approach, we separate the tasks. Resource-intensive but sporadic tasks run externally. Meanwhile, tasks that need constant maintenance run locally. As a result, we can achieve a cost-effective MLOps solution and keep the performance stable.

In this paper, we implement the proposed Hybrid MLOps in a private cloud environment with three interconnected virtual machines. By executing machine learning tasks using an actual pipeline, we confirm the effectiveness and feasibility of the method on a private cloud, showing that secure, reproducible, and resource-efficient ML experimentation is possible.

3. Challenge

Hybrid MLOps in a private cloud environment offer clear benefits for small-scale ML experimentation. However, they introduce several challenges that must be addressed to implement MLOps effectively.

〈3·1〉 Technical Difficulties in Hybrid Self-Hosted CI/CD

Deploying Hybrid MLOps pipelines requires expertise in system administration, networking, containerization, and ML frameworks. Self-hosted systems differ from cloud platforms in terms of infrastructure complexity. As a result, tasks such as dependency management, workflow orchestration, and network configuration must be manually handled. Improper setup can lead to pipeline failures, inconsistent results, or poor resource utilization⁽³⁾.

〈3·2〉 Hardware and Software Constraints

Small-scale hybrid environments often face limited CPU cores, memory, or GPU availability. Efficiently selecting tasks between CPU-only and GPU-enabled nodes is crucial to optimize performance, energy, and cost. Software versions, package dependencies, and container environments must also be carefully controlled to maintain reproducibility across heterogeneous resources.

〈3·3〉 Maintainability, Reproducibility, and Modularity

Long-term maintainability requires modular pipelines where preprocessing, training, evaluation, and deployment steps can be

independently updated or reused. Hybrid MLOps further demands precise logging of system configurations, hyperparameters, and resource usage to ensure reproducible experiments across on-premise and GPU-enabled worker nodes.

By addressing these challenges, a well-designed Hybrid MLOps system can provide a secure, reproducible, and resource-efficient infrastructure for small-scale ML experimentation. It balances performance, energy consumption, and cost, even within a private cloud environment.

4. Method

〈4·1〉 Pipeline Architecture

An overview of the proposed Hybrid MLOps infrastructure is shown in Figure 1. This system is composed of three primary components, distributed across heterogeneous resources to emulate realistic hybrid deployments:

- **GitLab VM (Control Plane)** - Hosts the GitLab instance, manages CI/CD pipeline triggers, stores workflow definitions, and archives artifacts. It also manages the code required for model training and deployment.
- **Worker VM (Compute Plane)** - Executes ML workflows, including environment provisioning, data preprocessing, model training, evaluation, and model export. The resources are assumed to be temporary and are created and cleaned up for each workflow. GPU-enabled resources are optionally leveraged to accelerate training.
- **On-Premise VM (Deployment & Storage Plane)** - Handles model reception, archiving, local deployment, and the product container group for inference in the production environment.

This separation clarifies the Development and Operation phases, enabling secure data handling, and modular pipeline execution, while supporting infrastructure-aware scheduling⁽¹⁾.

〈4·2〉 Workflow with GitLab CI/CD

The end-to-end workflow is orchestrated using GitLab CI/CD. Each commit to the repository triggers the pipeline which executes

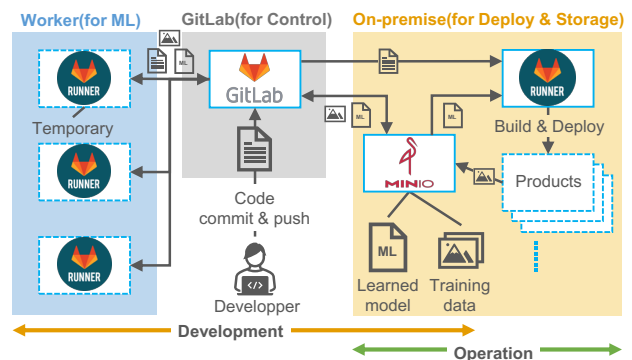


Fig. 1. System architecture

a series of automated steps:

This entire workflow is declaratively defined in a GitLab CI/CD configuration file (.gitlab-ci.yml), ensuring reproducibility and version control. Listing 1 shows a simplified excerpt of this file, illustrating the key stages and job definitions. As defined in this configuration, the pipeline consists of several automated stages. Figure 2 provides a conceptual overview of this workflow, from the initial commit trigger to the final model deployment. The key stages are detailed below:

In Figure 2, the Environment Setup stage establishes the required infrastructure. Both the on-premise virtual machine (VM) and the worker (VM) are provisioned with Docker and Docker Buildx to enable containerized execution. A MinIO instance is deployed on the on-premise VM for object storage management, and the MinIO client (mc) is installed on both environments to facilitate file transfer such as datasets or models. Additionally, Python 3.11 is verified on the worker VM, and all necessary dependencies are installed to ensure reproducibility and consistent

```
stages:
  - setup          # Environment setup stage
  - train          # Model training stage
# Setup environment and install dependencies
setup_environment:
  stage: setup
  ...
  tags: [worker]   # Run on a specific resource
  script:
  ...
# Train and evaluate the model
train_and_evaluate:
  stage: train
  image: python:3.12-slim
  tags: [worker]   # Run on a specific resource
  script:
    - . .venv/bin/activate
    - python3 pipeline/main.py
  artifacts:
    paths:
      - results_plots/
      - results/benchmarks.csv
  ...
```

Listing. 1. Excerpt of the CI/CD pipeline used in this study

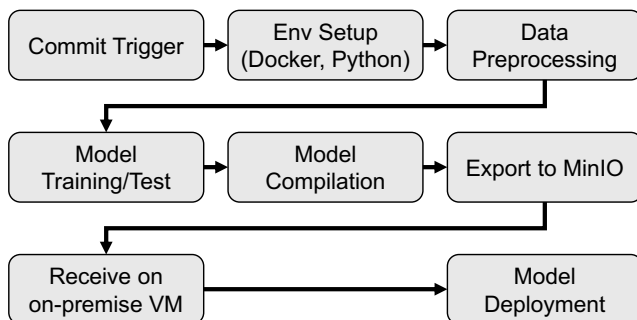


Fig. 2. CI/CD pipeline workflow

execution across experiments.

The Data Preprocessing phase involves cleaning, normalizing, and formatting the input datasets within Python scripts executed on the worker VM. This step prepares the data for subsequent model training.

During the Model Training phase, machine learning models are developed using Python and PyTorch. Convolutional Neural Networks (CNNs) and Deep Neural Networks (DNNs) are implemented as proof-of-concept architectures in locally defined Python files.

The Model Compilation and Export stage serializes trained models, containerizes them via Docker Buildx, and export them from the worker VM to the on-premise VM via MinIO using the mc client. This approach ensures consistency in storage and accessibility across the infrastructure.

Finally, the Deployment on On-Premise VM stage retrieves, archives, and registers the models as artifacts for future inference tasks. Temporary files generated during training are removed, while previously stored models on the MinIO server are either archived or deleted according to retention policies.

⟨4·3⟩ Model Training Phase

The training phase focuses on flexibility and reproducibility:

- **Datasets** – CIFAR-10 and MNIST are used as benchmarks.
- **Architectures** – Convolutional Neural Networks (CNNs) for image classification and Deep Neural Networks (DNNs) for dense data evaluation.
- **Hyperparameter Management** – Key hyperparameters (CPU core, RAM size, PyTorch thread, batch size, optimizer type) are managed through a YAML configuration file to facilitate reproducibility.
- **Logging and Metrics** – Training loss, accuracy, and other evaluation metrics are automatically logged for later analysis.

This modular approach ensures that each step can be independently modified, reused, or extended for future experiments, while minimizing manual intervention and maintaining reproducibility across environments.

5. Evaluation Setup

⟨5·1⟩ Experimental Setup

The experiments were conducted on the three-VM infrastructure described in Section 4. Each VM has the following specifications:

- **GitLab VM:** Intel Xeon (Cooperlake) E5-2620 2.10 GHz 6 cores, DDR3 15GB RAM, Ubuntu 22.04.5 LTS, Docker 28.0.1, no dGPU.
- **Worker VM:** AMD Ryzen 7 5800XT 8 cores, DDR4 31GB RAM, NVIDIA GeForce RTX 4060 8GB, Ubuntu

24.04.1 LTS, Docker 28.0.1.

- **On-Premise VM:** AMD EPYC-Milan 2 cores, DDR4 3.8GB RAM, Ubuntu 24.04.2 LTS, Docker 28.1.1+1, no dGPU.

All VMs are interconnected via GitLab and GitLab Runner. The GitLab CI/CD instance orchestrates the end-to-end ML pipeline, including environment setup, preprocessing, training, and model deployment.

〈5·2〉 Datasets

Two widely used benchmark datasets were employed to evaluate the pipeline:

- **MNIST** – Handwritten digit dataset with 60,000 training and 10,000 test images of size 28x28 pixels.
- **CIFAR-10** – 10-class image dataset containing 50,000 training and 10,000 test images of size 32x32 pixels.

These datasets were chosen to test the pipeline under both simple and moderately complex image classification tasks.

〈5·3〉 Models

Two types of models were implemented and trained:

- **Convolutional Neural Network (CNN):** Designed for image classification tasks, particularly suitable for spatially correlated input such as CIFAR-10 images. The CNN consists of two convolutional layers with 32 and 64 filters of size 3x3, each followed by a ReLU activation and 2x2 max pooling. This is connected to two fully connected layers with 128 and 10 output units, respectively, for classification. Dropout and batch normalization were not applied in this simple architecture to reduce computational overhead.
- **Deep Neural Network (DNN):** A fully connected architecture designed for dense inputs such as MNIST grayscale images. The DNN consists of three linear layers with 256, 128, and 10 units, respectively, each followed by ReLU activations. The input is flattened to a vector of length 784 before feeding into the first linear layer. This simple DNN allows evaluation of the pipeline on non-convolutional architectures.

〈5·4〉 Evaluation Metrics

To assess both model performance and infrastructure efficiency, the following metrics were employed:

- **Accuracy:** Standard measure of model performance on the MNIST and CIFAR-10 datasets.
- **Energy Consumption:** Total energy consumed by each model during training on each dataset.
- **Usage Cost:** The electricity cost calculated from energy consumption, and the estimated cost for executing the same task on an external cloud service.

- **Runtime:** Total time required for training each model on each dataset.

These metrics provide a quantitative basis for evaluating the effectiveness of the models as well as the efficiency and resource utilization of Hybrid MLOps in a private cloud environment.

6. Results

〈6·1〉 Model Performance

The DNN and CNN models were evaluated on the MNIST and CIFAR-10 datasets, respectively. Figures 3 and 4 show the training accuracy per epoch for both models. These curves show convergence and provide insight into the stability of training under the proposed Hybrid MLOps architecture.

In Figure 3, given the simplicity of the MNIST dataset, the Hybrid MLOps pipeline shows the capability to efficiently train the model, either with CPU or GPU.

In Figure 4, the more complex CIFAR-10 classification task, the training was also successful, showing that both CPU and GPU configurations reached a sufficient accuracy with a general convergence pattern.

The convergence patterns observed across both datasets validate the stability and efficiency of the training process within the implemented infrastructure framework⁽⁶⁾⁽⁷⁾.

〈6·2〉 Cost Analysis

The energy consumption associated with model training was

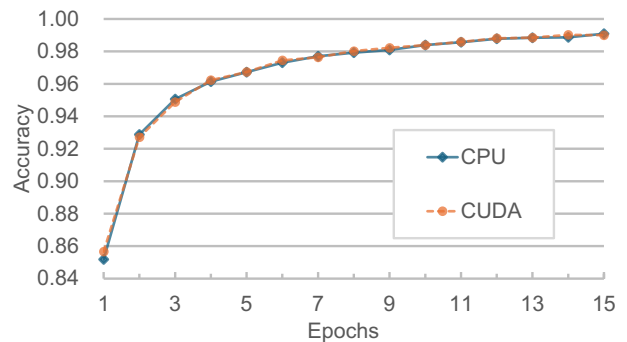


Fig. 3. Training accuracy of MNIST

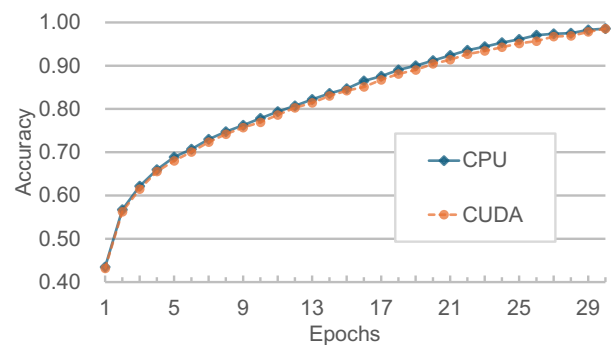


Fig. 4. Training accuracy of CIFAR-10

systematically measured on the Worker VM for both CNN and DNN architectures using the MNIST and CIFAR-10 datasets. Based on this energy consumption and runtime, we calculated the electricity cost for executing each task in the private cloud environment and the estimated cost for running it on a public cloud. The electricity cost is calculated based on the TEPCO (Tokyo Electric Power Company) rate structure as of July 2025, while the cloud cost is an estimate based on standard CPU and GPU VM models on Microsoft Azure.

Table 1 summarizes the observed training durations, energy consumption, and estimated costs. The data indicate that the private cloud is more cost-effective than the public cloud. Furthermore, it was shown that for CIFAR-10, the GPU (CUDA) has a clear advantage in cost-efficiency, whereas for MNIST, the difference is negligible.

〈6·3〉 Key Observations

From the experiments, several important observations can be drawn:

- (1) **Feasibility of ML Workflow:** The proposed method showed the ability to correctly execute the ML pipeline regardless of the data source (MNIST or CIFAR-10), training method, or computational architecture (CPU or GPU).
- (2) **Cost-Efficiency:** It was shown that Hybrid MLOps executed in a private cloud environment can be more cost-effective than using public cloud services, provided that the learning task fits within the resource constraints of the private environment.
- (3) **Dataset Sensitivity:** Lightweight datasets such as MNIST show minimal sensitivity to hardware configuration, with nearly equivalent training times and energy usage across CPU and GPU runs.

These findings underscore the importance of selecting appropriate hardware based on the dataset size and computational complexity, and provide empirical support for the deployment of Hybrid MLOps infrastructures that leverage local GPU resources to balance performance, cost, and energy efficiency.

Table 1. Training time, Energy consumption and cost comparison for each dataset.

Data set	Hardware	Train Time(s)	Energy (Wh)	Electric Price(\$)	Azure Price(\$)
MNIST	CPU	48.03	6.7	0.002	0.006
MNIST	CUDA	48.82	6.8	0.002	0.050
CIFAR-10	CPU	724.14	100.6	0.024	0.096
CIFAR-10	CUDA	93.87	13	0.003	0.096

7. Discussion

The experimental evaluation shows that the proposed Hybrid MLOps architecture is effective for practical machine learning workflows. The results highlight both the practical advantages of Hybrid MLOps in resource-constrained environments and the limitations that should guide future improvements.

〈7·1〉 Practical advantages

- **Modularity and Flexibility:** The CI/CD pipeline architecture enables independent modification, extension, and reuse of individual components, including data preprocessing, model training, and deployment. This modularity facilitates rapid experimentation and reduces debugging complexity, making the infrastructure adaptable to a variety of ML tasks.
- **Reproducibility:** By formalizing environment setup, dependencies, and hyperparameters through YAML-based orchestration, experiments can be consistently reproduced across multiple runs and virtual machines, addressing a common limitation of small-scale or self-hosted MLOps.
- **Data Security and Local Control:** Sensitive datasets remain primarily on-premise. GPU acceleration is selectively applied during training, ensuring that high-performance computation does not compromise data sovereignty.
- **Cost-Efficiency:** The results from the energy consumption and cost calculations indicate that for small-scale models, Hybrid MLOps on a private cloud can be more cost-effective. Furthermore, the efficiency difference between CPU and GPU can be significant depending on the learning target (e.g., CIFAR-10). This suggests that by dynamically selecting the configuration—such as whether to use public or private cloud resources and whether to use a CPU or GPU architecture—based on the task's complexity and scale, both energy and cost efficiency can be optimized.

〈7·2〉 Limitations and Challenges

- **Scalability Constraints:** While suitable for SMEs, the current infrastructure may face performance limitations with large datasets, high-resolution images, or deep learning models requiring extensive GPU resources. Scaling this approach may necessitate additional hardware or integration with external cloud resources.
- **Dataset and Model Scope:** The experimental evaluation was limited to MNIST and CIFAR-10 datasets, which are relatively small and structured. The generalizability of the infrastructure to more complex domains, such as natural language processing or high-resolution image recognition, remains untested.
- **Hardware and Performance Boundaries:** Limited CPU

cores, RAM, and GPU memory in the private cloud impose restrictions on training throughput. While acceptable for lightweight workflows, more demanding tasks may experience bottlenecks that could reduce the benefits of self-hosted MLOps.

⟨7·3⟩ Implications for Hybrid MLOps

The results emphasize that hybrid MLOps approaches can meaningfully balance performance, energy consumption, and cost:

- **Choice of Diverse Computational Resources:** Since Hybrid MLOps can be used in a wide range of configurations, it allows for the flexible use of resources across local, private cloud, and multiple public clouds, enabling optimal operation tailored to user requirements.
- **Energy- and Cost-Aware Scheduling:** By integrating runtime and energy monitoring into the pipeline, developers can make informed decisions about resource allocation and task placement, supporting sustainability and cost-conscious operations.
- **Feasibility in Resource-Constrained Environments:** Even with hardware in a private environment, Hybrid MLOps can achieve stable, reproducible training and evaluation, showing the practical applicability for small laboratories and SMEs.

8. Related Work

MLOps has become essential for streamlining ML development, deployment, and maintenance. Kreuzberger et al. (2022) provide an overview of MLOps, emphasizing modular pipelines, continuous integration, and continuous delivery⁽¹⁾. Hybrid deployments introduce additional complexity: Banerjee et al. (2020) highlight challenges in orchestrating heterogeneous resources while maintaining reproducibility and performance⁽³⁾.

Energy efficiency and sustainability are increasingly critical. Sánchez-Mompó et al. (2025) quantify energy costs in AI workflows⁽⁴⁾, and Li (2024) analyzes energy-efficient strategies for deep convolutional networks⁽⁵⁾. To improve cost-efficiency for smaller organizations, Hiramí et al. (2023) propose hybrid MLOps that combine cloud and on-premises resources while maintaining performance⁽²⁾.

Finally, standardized benchmarks like UltraMNIST (Gupta et al., 2024) and experimental studies (Zhu et al., 2018) support reproducible evaluation of training efficiency across infrastructures⁽⁶⁾⁽⁷⁾. Building on these works, this paper proposes a practical, cost- and energy-aware hybrid MLOps system for small-scale organizations.

9. Conclusion

In this work, we presented a lightweight, self-hosted CI/CD

infrastructure for machine learning workflows, tailored for small-scale laboratories and enterprises. Leveraging three interconnected virtual machines, the system automates data preprocessing, model training, deployment, and archival while ensuring reproducibility, modularity, and data sovereignty.

Experimental evaluation using CNN and DNN models on the MNIST and CIFAR-10 datasets showed that the proposed infrastructure can reliably support model training and evaluation. The results also showed that cost-efficiency varies depending on the choice of GPU versus CPU, as well as public versus private cloud. Lightweight tasks such as MNIST classification derive little benefit from GPU acceleration, and for the tasks conducted in this experiment, using the local GPU in the private cloud was more cost-effective than a cloud GPU. These observations emphasize the importance of task-specific hardware selection and cost-aware workflow orchestration in Hybrid MLOps.

The study further confirms the practical applicability of Hybrid MLOps principles in small-scale, self-hosted setups. By integrating modular, reproducible pipelines with performance, cost, and energy monitoring, the infrastructure provides a feasible alternative to fully cloud-based solutions for organizations with budgetary, regulatory, or data-sensitivity constraints.

Future work will focus on extending the infrastructure to support larger datasets, more complex models, and high-resolution inputs, as well as incorporating multi-GPU or distributed training.

References

- (1) Dominic Kreuzberger, Niklas Kühl, and Sebastian Hirschl: "Machine Learning Operations (MLOps): Overview, Definition, and Architecture", arXiv preprint arXiv:2205.02302 (2022).
- (2) Shuji Hiramí, et al.: "Proposal of Hybrid MLOps Infrastructure toward Expanding Machine Learning for Small and Medium Size Organizations by Reducing Costs", Proceedings of the 85th National Convention of the Information Processing Society of Japan (IPJSJ) (2023).
- (3) Amitabha Banerjee, et al.: "Challenges and Experiences with MLOps for Performance Diagnostics in Hybrid-Cloud Enterprise Software Deployments", Proceedings of the 2020 USENIX Annual Technical Conference (ATC 20) (2020).
- (4) Adrián Sánchez-Mompó, et al.: "Green MLOps to Green GenOps: An Empirical Study of Energy Consumption in Discriminative and Generative AI Operations", Information, Vol. 16, No. 4, p. 281 (2025).
- (5) D. Li: "Evaluating the Energy Efficiency of Deep Convolutional Neural Networks", Sustainability Computing: Informatics and Systems (2024).
- (6) Deepak K. Gupta, et al.: "An UltraMNIST classification benchmark to train CNNs for very large images", Scientific Data, Vol. 11, No. 1, p. 771 (2024).
- (7) Hongyu Zhu, Mohamed Akrouf, and Bojian Zheng: "Benchmarking and Analyzing Deep Neural Network Training", Proceedings of the 2018 IEEE International Symposium on Workload Characterization (IISWC) (2018).
- (8) Lisana Berberi, et al.: "Machine learning operations landscape: platforms and tools", Artificial Intelligence Review, Vol. 58, No. 6, p. 167 (2025).